Resource Management: Token Usage and Cost Tracking with Langfuse

> **For the challenge, all costs are tracked exclusively via Langfuse session IDs.** This tutorial shows how to integrate Langfuse with LangChain to automatically track token usage, costs, and performance.

This tutorial teaches you how to **monitor and manage resources** when using AI agents with **Langfuse** as the observability platform. LangChain has a native Langfuse `CallbackHandler` that automatically captures token usage, costs, and latency.

Why Resource Management Matters

When building production AI agent systems, understanding resource usage is crucial:

- **Cost control** - LLM API calls cost money; you need to track spending
- **Performance optimization** - Token usage affects response times and costs
- **Budget planning** - Predict costs before scaling your system
- **Debugging** - Token metrics help identify inefficient patterns

What Are Tokens?

Tokens are the units that language models process. Roughly:

- 1 token ≈ 4 characters of English text
- 1 token ≈ 0.75 words
- 1000 tokens ≈ 750 words

When you call an agent, it uses:

- **Input tokens** - Your question + system prompt + conversation history
- **Output tokens** - The agent's response
- **Cache tokens** - Optional caching for faster/cheaper repeated queries

What You'll Learn

In this tutorial, you'll:

1. Set up Langfuse tracing for LangChain using `@observe()` and `CallbackHandler`
2. Understand how Langfuse automatically tracks token usage and costs
3. Use session IDs to group multiple calls under a single session
4. Track costs across multiple agent calls
5. Generate unique session IDs for the challenge

Prerequisites

Before starting, make sure you have:

- **Python 3.13** (suggested) installed (see warning below about Python 3.14)
- **OpenRouter API key** (get one free at https://openrouter.ai)
- **Langfuse credentials** — You should have received `LANGFUSE_PUBLIC_KEY`, `LANGFUSE_SECRET_KEY`, and `LANGFUSE_HOST` for the challenge
- **Completed Tutorial 01** - You should understand basic agent creation

> ⚠️ **Python 3.14 Warning:** Python 3.14 can cause compatibility issues with Langfuse. We recommend using **Python 3.13** to avoid problems.
>
> ⚠️ **Langfuse SDK Version:** We recommend using **Langfuse SDK v3** for maximum compatibility with the platform. Langfuse v4 is not fully supported and may cause unexpected issues.

> **First time?** See **Tutorial 01** for detailed instructions on installing Python, setting up a virtual environment, and creating your API key.

Quick Setup Checklist

1. **Install Python 3.13** (suggested): Download from python.org — verify with `python3 --version` (avoid Python 3.14)
2. **Create a virtual environment**: `python3 -m venv venv && source venv/bin/activate`
3. **Get an OpenRouter API key**: Sign up at openrouter.ai → Keys → Create Key
4. **Create a `.env` file** in the project root with:

```
OPENROUTER_API_KEY=your-api-key-here
LANGFUSE_PUBLIC_KEY=pk-your-public-key-here
LANGFUSE_SECRET_KEY=sk-your-secret-key-here
LANGFUSE_HOST=https://challenges.reply.com/langfuse
TEAM_NAME=your-team-name
LANGFUSE_MEDIA_UPLOAD_ENABLED=false
```

Installation

Install the required dependencies directly. This cell is self-contained—no external `requirements.txt` needed.

- `langfuse` provides the `@observe()` decorator and `CallbackHandler` for automatic tracing
- `ulid-py` generates unique session IDs

In [ ]:
```
%pip install langchain langchain-openai "langfuse>=3,<4" python-dotenv ul
```

Setup Model

Import the necessary libraries and configure the model. This is the same setup from Tutorial 01.

In [ ]:
```
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage

# Load environment variables from .env file
load_dotenv()

# Chosen model identifier
model_id = "gpt-4o-mini"

# Configure OpenRouter model
model = ChatOpenAI(
    api_key=os.getenv("OPENROUTER_API_KEY"),
    base_url="https://openrouter.ai/api/v1",
    model=model_id,
    temperature=0.7,
    max_tokens=1000,
)

print(f"✓ Model configured: {model_id}")
```

Initialize Langfuse and Helper Functions

How Langfuse Works with LangChain

The integration combines two mechanisms:

1. `@observe()` **decorator** - Wraps a function to automatically create a Langfuse **trace** on each call. All Langfuse operations inside the decorated function are nested under that trace.
2. `CallbackHandler()` - Created *inside* the `@observe()` function, it automatically attaches to the current trace and captures LangChain-specific metrics (tokens, costs, latency).
3. **Session tracking** - Multiple calls can be grouped under the same `session_id` by passing `config={"metadata": {"langfuse_session_id": session_id}}` to LangChain calls. This lets you group all calls from a single run together.
4. **Unique session IDs** - Generated with `ulid` in the format `{TEAM_NAME}-{ULID}` for easy identification. `session_id` must not contain blank spaces; normalize `TEAM_NAME` by replacing spaces with `-` when building the ID (your team name in other contexts may still include spaces).

What Gets Tracked Automatically

The `CallbackHandler` captures:

- **Inputs and outputs** - All messages sent to and received from the model

- **Token usage** - Input, output, and cache tokens (when available)
- **Costs** - Automatically calculated based on model pricing
- **Latency** - Time taken for each operation
- **Metadata** - Model parameters, temperature, etc.

In [ ]:
```
import ulid
from langfuse import Langfuse, observe
from langfuse.langchain import CallbackHandler

# Initialize Langfuse client
langfuse_client = Langfuse(
    public_key=os.getenv("LANGFUSE_PUBLIC_KEY"),
    secret_key=os.getenv("LANGFUSE_SECRET_KEY"),
    host=os.getenv("LANGFUSE_HOST", "https://challenges.reply.com/langfus
)

def generate_session_id():
    """Generate a unique session ID using TEAM_NAME and ULID."""
    # session_id must not contain blank spaces; TEAM_NAME may include spa
    team = os.getenv("TEAM_NAME", "tutorial").replace(" ", "-")
    return f"{team}-{ulid.new().str}"

def invoke_langchain(model, prompt, langfuse_handler, session_id):
    """Invoke LangChain with the given prompt and Langfuse handler."""
    messages = [HumanMessage(content=prompt)]
    response = model.invoke(
        messages,
        config={
            "callbacks": [langfuse_handler],
            "metadata": {"langfuse_session_id": session_id},
        },
    )
    return response.content

@observe()
def run_llm_call(session_id, model, prompt):
    """Run a single LangChain invocation and track it in Langfuse."""
    # Pass session_id via LangChain metadata for session grouping
    # Create Langfuse callback handler for automatic generation tracking
    # The handler will attach to the current trace created by @observe()
    langfuse_handler = CallbackHandler()

    # Invoke LangChain with Langfuse handler to track tokens and costs
    response = invoke_langchain(model, prompt, langfuse_handler, session_

    return response

print("✓ Langfuse initialized successfully")
print(f"✓ Public key: {os.getenv('LANGFUSE_PUBLIC_KEY', 'Not set')[:20]}.
print("✓ Helper functions ready: generate_session_id(), invoke_langchain(
```

Run a Single Traced Call

Now we'll use `run_llm_call()` - decorated with `@observe()` - to make a traced call. Here's what happens under the hood:

1. `@observe()` creates a new Langfuse trace when the function is called
2. `metadata.langfuse_session_id` tags each call with our session ID so all calls in this run are grouped together
3. `CallbackHandler()` is created *inside* the decorated function, so it automatically attaches to the current trace
4. **Token usage, costs, and latency** are all captured automatically

In [ ]:
```
session_id = generate_session_id()
print(f"Session ID: {session_id}\n")

response = run_llm_call(session_id, model, "What is the square root of 14

print(f"\nInput:    What is the square root of 144?")
print(f"Response: {response}")

langfuse_client.flush()

print(f"\n✓ Trace sent to Langfuse with full token usage and cost data")
print(f"✓ Grouped under session: {session_id}")
print("✓ You can view this session on the Langfuse dashboard in the platf
```

Track Multiple Calls with Session Grouping

Since every call to `run_llm_call()` shares the same `session_id`, all traces are grouped together. There's no need to manually accumulate tokens — Langfuse aggregates everything for you.

This is the key advantage over manual tracking:

- **No manual accumulation** — Langfuse sums tokens and costs across all traces in a session
- **Automatic cost calculation** — Based on Langfuse's built-in model pricing
- **Queryable traces** — You (or the organizers) can query Langfuse by `session_id` using the Langfuse Python client or HTTP API to see usage and costs

Let's make multiple calls under the same session:

In [ ]:
```
questions = [
    "What is machine learning?",
    "Explain neural networks briefly.",
    "What is the difference between AI and ML?"
]

session_id = generate_session_id()
print(f"Session ID: {session_id}")
print(f"Making {len(questions)} agent calls with Langfuse tracing...\n")

for i, question in enumerate(questions, 1):
    response = run_llm_call(session_id, model, question)
    print(f"Call {i}: {question[:40]}...")
    print(f"  Response: {response[:80]}...\n")

langfuse_client.flush()
```

```
print("=" * 50)
print(f"✓ All {len(questions)} traces sent to Langfuse!")
print(f"✓ All grouped under session: {session_id}")
print("✓ You can view this session on the Langfuse dashboard in the platf
```

Viewing Your Traces

You can view your tracing details on the **Langfuse dashboard** available in the platform page. The dashboard is associated with your team, so all traces generated by your team members will be visible there.

On the dashboard you can see details about **sessions**, **traces**, and **observations**.

Note: The dashboard is **not updated in real time**. There may be a delay of a few minutes before the latest traces appear.

What You've Learned

Congratulations! You've learned how to monitor and manage resources for LangChain agents using Langfuse. Here's what we covered:

✅ **Langfuse setup** — Initializing the client, `@observe()` decorator, and `CallbackHandler` for automatic tracing
✅ **Automatic token tracking** — `CallbackHandler` captures all token usage from LangChain calls
✅ **Automatic cost calculation** — Langfuse calculates costs based on its built-in model pricing
✅ **Session grouping** — Using `metadata.langfuse_session_id` to group calls
✅ **Session ID generation** — Creating unique IDs with `{TEAM_NAME}-{ULID}` format (`session_id` has no spaces; spaces in `TEAM_NAME` become `-`)
✅ **Trace viewing** — Checking your tracing details on the Langfuse dashboard in the platform page (associated with your team)

Key Takeaways

1. **`@observe()` + `CallbackHandler`** — The recommended pattern: decorate functions with `@observe()` and create `CallbackHandler()` inside to automatically attach to the current trace
2. **Session tracking** — Use `generate_session_id()` with ULID and `metadata.langfuse_session_id` to group calls
3. **`langfuse_client.flush()`** — Always flush after your calls to ensure all traces are sent
4. **No manual cost tracking needed** — Langfuse handles token counting and cost calculation automatically
5. **Check your traces** — View your tracing details on the Langfuse dashboard in the platform page (note: there may be a few minutes of delay)

How LangChain + Langfuse Tracing Works

```
@observe() decorated function
    ↓
Creates Langfuse trace → pass metadata.langfuse_session_id
    ↓
CallbackHandler() attaches to current trace
    ↓
model.invoke(messages, config={"callbacks": [handler]})
    ↓
CallbackHandler captures: tokens, costs, latency, I/O
    ↓
langfuse_client.flush() → sends to Langfuse
    ↓
Langfuse dashboard (platform page) → view sessions, traces
and observations
```

Cost Optimization Strategies

When building production systems:

- **Monitor regularly** — Check the Langfuse dashboard to review costs after each session
- **Choose models wisely** — Balance cost vs. capability for your use case
- **Optimize prompts** — Shorter prompts = fewer input tokens = lower costs
- **Start small, scale up** — Begin with smaller models and only switch to larger ones if needed
- **Multi-agent strategy** — Use larger models for critical decisions and smaller models for simpler tasks

Best Practices

1. **Always set session IDs** — Essential for the challenge; groups all costs under one session
2. **Use `@observe()` + `CallbackHandler`** — Wrap LLM-calling code so Langfuse captures everything automatically
3. **Flush after calls** — Call `langfuse_client.flush()` to ensure traces are sent
4. **Generate unique session IDs** — Use ULID to avoid collisions

Next Steps

Now that you understand resource management with Langfuse, you can:

- **Apply to your agents** — Add Langfuse tracing to your own systems
- **Optimize existing agents** — Use the Langfuse dashboard to identify and reduce expensive operations
- **Build cost-aware systems** — Design agents with cost efficiency in mind
- **Scale confidently** — Understand costs before deploying at scale

For more information, visit the Langfuse documentation.